# SMART CAMPUS PLACEMENT & SKILL GAP ANALYZER

Frontend Roadmap, Page-by-Page Specification & Implementation Plan

A practical blueprint for building the student workflow first, then extending the platform for TPO/admin and recruiter roles.

| | |
|---|---|
| **Recommended stack** | Next.js App Router, React, TypeScript, Tailwind CSS |
| **Core workflow** | Login → Resume upload → Select target job → NLP/semantic matching → Skill-gap report |
| **Primary users** | Students, Training & Placement Officer (TPO)/Admin, recruiters if included in scope |
| **Design direction** | Professional academic SaaS: navy, blue, teal and off-white; clear hierarchy and restrained visuals |

**Build principle: Make the complete resume-to-analysis workflow functional before adding optional analytics, advanced animations or recruiter features.**

# 1. Roadmap at a Glance

Build the product in four phases. The first two deliver the core student experience; the remaining phases improve reliability and add role-specific features.

| Phase | Scope | Exit criteria |
|---|---|---|
| **1. Foundation** | Project setup, design system, navigation, landing page, login and registration. | App shell and authentication screens work; integration is planned or connected. |
| **2. Core student workflow** | Dashboard, resume upload, job selection, analysis report and API integration. | Student uploads a resume, selects a job, runs analysis and sees backend results. |
| **3. Persistence & reliability** | Report history, profile, validation, loading/empty/error states, permissions and testing. | Reports persist, errors are understandable, and access is protected. |
| **4. Extensions** | TPO/admin tools, batch analytics, recruiter views and report export. | Only in-scope features are enabled and tested. |

## Recommended build order

- Landing page and authentication screens
- Shared layout and reusable buttons, cards, inputs and navigation
- Student dashboard
- Resume upload and parsing status
- Job search and job details
- Analysis report with matched skills, missing skills and recommendations
- Report history and profile
- TPO/admin, then recruiter features if required

**MVP boundary: Complete pages 1–6 end-to-end first. Admin and recruiter pages are extensions and should not delay the main ML workflow.**

# 2. Page Inventory — 8 Main Pages

Some pages have related subpages, such as a job detail page or a report detail page.

| No. | Page | User | Content displayed | Layout & theme |
|---|---|---|---|---|
| 1 | **Landing page** | Public | Platform intro, features, how it works, Login/Get Started actions. | Spacious hero and feature sections; white/off-white, navy, blue CTA, teal accents. |
| 2 | **Login & registration** | Authentication | Login: email/student ID, password, validation; optional reset. Registration: name, university email/ID, password fields if self-registration is allowed. | Centered form card; calm background, navy headings, blue buttons and clear errors. |
| 3 | **Student dashboard** | Student | Resume status, latest match score, target roles, missing-skill summary, Start Analysis and recent reports. | Sidebar + top header, responsive summary cards, main action and recent-analysis list. |
| 4 | **Resume management** | Student | Upload PDF, accepted format/size, file details, parsing status, replace/remove if supported, extracted skills and parser errors. | Upload panel first, then file status and extracted-skill section. |
| 5 | **Job/company selection** | Student | Search/filter jobs; title, company, description, required skills, eligibility if available, details and Analyze action. | Search/filter row, job cards, details view and prominent analysis CTA. |
| 6 | **Skill-gap report** | Core | Match score, matched skills, missing/weak skills, resume-vs-job comparison, recommendations, job details, date and optional export. | Report header, score cards, green matched tags, amber gap tags, comparison and recommendation sections. |
| 7 | **TPO/admin dashboard** | Admin extension | Overview counts, student directory, job descriptions, eligibility rules, batch skill-gap analytics and reports. | Admin sidebar, searchable tables, filters, metric cards and simple charts. |
| 8 | **Recruiter dashboard** | Recruiter extension | Company profile, job postings, eligible candidates, candidate skill-match comparison and permitted profile views. | Recruiter sidebar, job-management table, candidate search and ranked/matched candidate list. |

**Priority: pages 1–6 first. Pages 7–8 depend on project scope and available time.**

# 3. Detailed Page Specifications

Pages 1–4: public entry, account access and the start of the student workflow.

## Page 1 — Landing page

- **Navigation:** logo, Home, Features, How It Works, Login and Get Started.
- **Hero:** “Know Your Skills. Prepare for Your Dream Role.” with short explanation and Analyze My Resume CTA.
- **How it works:** upload resume → choose job → analyze match → improve skills.
- **Features:** skill extraction, semantic matching, skill-gap reports and learning recommendations.
- **Footer:** About, Contact, Privacy Policy and institution/project details.

Theme: white/off-white background, navy headings, blue primary buttons and teal accents. Avoid excessive animation.

## Page 2 — Login and registration

- **Login:** email/student ID, password, show/hide password, Login, validation and optional Forgot Password.
- **Registration:** name, university email/student ID, password and confirmation; only if self-registration is allowed.
- **Routing:** send users to the dashboard for their role; block unauthorized access to protected pages.

Theme/layout: centered form on a light background; clear labels, keyboard-accessible controls and readable error messages.

## Page 3 — Student dashboard

- **Sidebar:** Dashboard, My Resume, Find a Job, Skill Gap Analysis, Reports, Profile and Logout.
- **Header:** welcome message, profile menu and notifications only if implemented.
- **Summary cards:** resume status, latest match score, target roles and missing-skill summary.
- **Primary action:** Start a New Skill-Gap Analysis.
- **Recent analyses:** role/job, date, score and View Report.

Theme/layout: persistent sidebar and top header; responsive summary cards; recent analyses below. Use “Not analyzed yet” rather than invented values.

## Page 4 — Resume management

- Drag-and-drop upload area and Browse Files; enforce backend-configured file types and size limits.
- Show file name/date, upload progress and status: Not uploaded → Uploading → Processing → Ready; keep Error separate.
- Provide Replace/Remove only if supported.
- Show extracted skill tags and clear errors for unsupported or unreadable files.

Theme/layout: upload panel first, then file details and extracted skills. Display only skills actually extracted from the student's resume.

# 4. Detailed Page Specifications (Continued)

Pages 5–8: target selection, model output and optional role-specific dashboards.

## Page 5 — Job/company selection

- Search by job title/company; show only filters supported by the dataset.
- Job cards show title, company, summary, required skills, eligibility if available, View Details and Analyze My Resume.
- Job detail shows full description, required skills, eligibility and Analyze Against This Job.
- Optional: paste a custom job description in a clearly separate flow.

Theme/layout: search and filters above job cards; detail view has a prominent analysis action.

## Page 6 — Skill-gap report (core feature)

- **Header:** target job/company, resume file and analysis date.
- **Summary:** Skill Match Score and matched/required skill counts.
- **Matched skills:** resume skills aligned with job requirements.
- **Missing/weak skills:** relevant job requirements not adequately identified in the resume.
- **Comparison:** resume skills versus job requirements, using a clear list or table.
- **Recommendations:** courses, resources, project ideas or truthful resume improvements, where supported.
- **Actions:** choose another job, revisit history and download/print only if implemented.

Theme/layout: report header, score cards, green matched tags, amber gap tags, comparison and recommendation sections. Never label semantic similarity as probability of getting hired; scores must come from the actual model/backend.

## Page 7 — TPO/admin dashboard (extension)

- Sidebar: Overview, Student Directory, Job Descriptions, Eligibility Rules, Skill-Gap Analytics, Reports and Settings.
- Overview: real counts for students, resumes, jobs and analyses.
- Student directory: search/filter and permitted student/eligibility details.
- Job management: create/edit descriptions and required skills.
- Batch analytics: common skill gaps and training needs.

Theme/layout: enterprise dashboard with metric cards, searchable tables, filters and simple charts. Protect student data and enforce admin access.

## Page 8 — Recruiter dashboard (extension)

- Company/recruiter profile and job posting management.
- Eligible candidates and skill-match comparisons for a selected job.
- Search/filter candidates and show only authorized profile information.
- Keep candidate ranking explainable and tied to implemented criteria.

Theme/layout: same base colors with denser job tables, candidate lists and search/filter controls. Build only if recruiter features are in scope.

# 5. Shared Theme & Design System

Use one coherent design system across all roles. Dashboards may differ in information density, but should feel like one product.

| Token | Hex | Use |
|---|---|---|
| **Navy blue** | #17365D | Headings, key text and navigation accents |
| **Blue** | #2563EB | Primary buttons, links and active navigation |
| **Teal** | #0F766E | Secondary accents and positive emphasis |
| **Off-white** | #F8FAFC | Page background and whitespace |
| **Green** | #16A34A | Matched skills and success states |
| **Amber** | #D97706 | Missing skills, warnings and attention states |
| **Red** | #B42318 | Errors and destructive actions, used sparingly |

## Component rules

- **Typography:** Inter or system sans-serif with clear heading levels.
- **Cards:** white surfaces, subtle borders, 10–14 px corners and restrained shadows.
- **Spacing:** consistent 8 px spacing scale.
- **Buttons:** blue primary and outlined secondary; include focus, disabled and loading states.
- **Icons:** one consistent Lucide icon set.
- **Responsive:** desktop sidebar; collapsible navigation on small screens; stack cards on mobile.
- **Accessibility:** visible labels, keyboard focus, contrast, and text/icons in addition to color.

## Visual hierarchy

| Element | Treatment |
|---|---|
| **Page title** | Large navy heading with contextual subtitle |
| **Primary action** | Blue button with action-oriented text |
| **Supporting information** | Muted but readable text |
| **Positive result** | Green indicator plus “Matched” label |
| **Skill gap/warning** | Amber indicator plus explanatory text |

# 6. Recommended Frontend File Structure

Use Next.js App Router with TypeScript and Tailwind CSS. Pages, reusable UI and API communication are kept separate.

```
smart-campus-placement/
■■■ public/images/ (logo, static illustrations)
■■■ src/
■ ■■■ app/
■ ■ ■■■ layout.tsx; globals.css; page.tsx
■ ■ ■■■ (auth)/login/page.tsx; register/page.tsx
■ ■ ■■■ (student)/student/
■ ■ ■ ■■■ layout.tsx
■ ■ ■ ■■■ dashboard/page.tsx
■ ■ ■ ■■■ resume/page.tsx
■ ■ ■ ■■■ jobs/page.tsx; jobs/[jobId]/page.tsx
■ ■ ■ ■■■ analysis/page.tsx; analysis/[reportId]/page.tsx
■ ■ ■ ■■■ reports/page.tsx; profile/page.tsx
■ ■ ■■■ (admin)/admin/
■ ■ ■ ■■■ layout.tsx; dashboard/page.tsx
■ ■ ■ ■■■ students/page.tsx; jobs/page.tsx; analytics/page.tsx
■ ■ ■■■ (recruiter)/recruiter/
■ ■ ■ ■■■ layout.tsx; dashboard/page.tsx; jobs/page.tsx
■ ■ ■■■ unauthorized/page.tsx
■ ■■■ components/
■ ■ ■■■ ui/ (Button, Input, Card, Badge, Modal)
■ ■ ■■■ layout/ (Navbar, Sidebar, PageHeader, Footer)
■ ■ ■■■ dashboard/ (SummaryCard, RecentAnalyses)
■ ■ ■■■ resume/ (ResumeUploader, ResumePreview, ExtractedSkills)
■ ■ ■■■ jobs/ (JobCard, JobSearch, JobDetails)
■ ■ ■■■ analysis/ (MatchScore, MatchedSkills, MissingSkills,
■ ■ SkillsComparison, Recommendations)
■ ■■■ services/ (api, auth, resume, jobs, analysis, admin)
■ ■■■ hooks/ (useAuth, useResume)
■ ■■■ types/ (student, resume, job, analysis)
■ ■■■ lib/ (utils, validation)
■■■ .env.local; package.json; tsconfig.json; README.md
```

## Folder responsibilities

| Folder | Purpose |
|---|---|
| **app/** | URLs, pages and role-specific layouts |
| **components/ui/** | Reusable buttons, inputs, cards, badges and dialogs |
| **components/resume/, jobs/, analysis/** | Feature-specific UI components |
| **services/** | API requests to the backend |
| **hooks/** | Reusable client-side state and behavior |
| **types/** | TypeScript definitions for API data |
| **lib/** | Shared utilities and form validation |

Route groups such as (student) organize code but do not appear in URLs. The dashboard URL is /student/dashboard.

# 7. Frontend–Backend Integration

The frontend collects input and presents results. FastAPI handles authentication, validation, parsing, semantic matching, persistence and response generation.

| Layer | Responsibility |
|---|---|
| **Next.js frontend** | Login, resume upload, job selection, loading states and report rendering |
| **FastAPI backend** | Authentication, validation, request routing and access control |
| **ML processing** | PDF/text parsing, skill extraction, embeddings, similarity and score generation |
| **PostgreSQL** | Student/job records and persisted reports, according to backend schema |
| **ChromaDB or FAISS** | Optional vector indexing/retrieval, depending on chosen architecture |

## Suggested API contract — adapt to the implemented backend

| Operation | Example endpoint | Purpose |
|---|---|---|
| Login | POST /api/auth/login | Verify credentials and establish session |
| Student profile | GET /api/students/me | Retrieve current profile |
| Upload resume | POST /api/resumes | Upload and start processing |
| Resume details | GET /api/resumes/{resumeId} | Read parsing status and extracted skills |
| List jobs | GET /api/jobs | Search available roles |
| Job details | GET /api/jobs/{jobId} | Read description and requirements |
| Run analysis | POST /api/analyses | Compare resume with target job |
| Get report | GET /api/analyses/{reportId} | Retrieve saved results |

Endpoint names are examples only. Align paths, payloads, response schemas and error codes with the actual FastAPI implementation.

## Environment configuration and security

- Use NEXT_PUBLIC_API_BASE_URL only for the public API base URL.
- Never expose database passwords, signing secrets or private credentials in NEXT_PUBLIC_ variables.
- Prefer properly configured HTTP-only cookies for authentication; enforce authorization on the backend.
- Validate file type and size on both frontend and backend; never trust client-side checks alone.

# 8. End-to-End Workflow & UI States

| Step | Student action | Frontend behavior | Backend/model work |
|---|---|---|---|
| **1** | Logs in | Validate form; show progress and errors | Verify credentials and establish session |
| **2** | Uploads resume | Show upload progress and parsing status | Validate file, extract text and identify skills |
| **3** | Selects role/company | Show job details and required skills | Retrieve job description and requirements |
| **4** | Starts analysis | Show analyzing state; prevent duplicate submissions | Compare skills and calculate configured score |
| **5** | Views report | Display matched/missing skills and recommendations | Return and optionally persist report |
| **6** | Chooses another job/report | Open new analysis or report history | Retrieve requested record under access rules |

## States every major page should handle

| State | Example | Expected UI |
|---|---|---|
| **Loading** | Parsing or analysis in progress | Progress message; prevent duplicate submission |
| **Empty** | No resume or reports yet | Helpful explanation and clear next action |
| **Error** | Invalid PDF, network failure, expired session | Specific message and safe retry/recovery |
| **Success** | Resume parsed or report generated | Confirmation and next step |
| **Unauthorized** | Student opens admin-only page | Block access and route appropriately |

## Quality checklist

- Test desktop and mobile layouts.
- Test valid, invalid and oversized resume uploads.
- Test empty dashboard and first-time user states.
- Test backend timeouts, API errors and session expiry.
- Ensure students cannot read other students' reports.
- Enforce admin/recruiter access server-side.
- Check keyboard navigation, labels, contrast and accessible status messages.
- Ensure match scores and skills come from actual backend output, not sample data.

# 9. Final Implementation Checklist

Mark an item complete only when it works in the running application, not merely when a screen has been designed.

## Phase 1 — Foundation

- ■ Create Next.js with TypeScript and Tailwind CSS.
- ■ Set global fonts, colors, spacing, buttons, cards and form styles.
- ■ Build landing, login and registration pages.
- ■ Create shared navigation and role-specific layouts.

## Phase 2 — Core student workflow

- ■ Build dashboard and useful empty states.
- ■ Implement resume upload and processing status.
- ■ Build job search, job cards and details.
- ■ Connect analysis endpoint and render actual results.
- ■ Show matched skills, missing skills and supported recommendations.

## Phase 3 — Persistence and reliability

- ■ Add profile, report history and report detail pages.
- ■ Implement loading, validation, error and retry states.
- ■ Verify authentication, authorization and data privacy.
- ■ Test API integration and responsive layouts.

## Phase 4 — Extensions

- ■ Build TPO/admin management and analytics if required.
- ■ Build recruiter features if in scope.
- ■ Add report export and learning links if implemented.
- ■ Document setup instructions and environment variables.

---

## Definition of a successful MVP

A student can log in, upload a resume, choose a target job, run the backend analysis and view a trustworthy report of matched skills and skill gaps—with clear loading/error states and protected access. Demonstrate this milestone before expanding the product.

End of roadmap